

[사내PC 실행용 · 8단계] 폴더 단위 demotion 계획 수립

소요 약 5분 · 읽기 전용 · 스킬 설치 불필요

회신은 ANS8 | 한 줄 + [P-3] 블록 붙여넣기.

0. 7단계 결과 요약과 이 단계의 목적

7단계에서 확인된 것:

관측	의미
intraDropped=0 (leaf)	SAM이 export 시점에 이미 동일 폴더 참조를 제외했다. 가설 확정
leaf 1 SCC / 46 folders	79개 중 46개 폴더가 하나의 거대 순환. 이게 MCD 페널티의 실체
파일 단위 5 SCC	fixer가 보던 것. 규모가 전혀 다르다
Utility 46/8	46개가 참조, 8개를 되참조. 그 8개가 순환의 원인

관측	의미
L1C/Common/Export 29/1	되참조 1개 남음. 기존 DB 폴 더로 추정

7단계 brk=0 은 잘못 읽힐 수 있다

brk 는 SCC 개수 감소만 썼다. 46노드 SCC에서 edge를 빼서 30노드로 줄어도

SCC 개수는 1 그대로라 brk=0 이 된다.

"단일 edge가 무의미하다"가 아니라 "그 지표가 못 잡는다"였다.

이 단계는 제대로 된 지표로 다시 센다 — 순환에서 빠져나오는 폴더 수.

1. 준비물

- sam_metrics_mcd_detail_mfs_relations.csv
(7단계와 같은 파일)
- Python 3.9+

2. STEP 1 — 스크립트 저장

mcd_demotion_plan.py 로 저장. UTF-8, 내용은 전부 ASCII.

```
#!/usr/bin/env python3
"""Folder-level demotion planner for SAM
MCD.

Read-only. Ranks folder-level actions by
REDUCTION IN FOLDERS-IN-CYCLE,
not by SCC count -- in one dense 46-folder
SCC, removing an edge can shrink
the component a lot while leaving the SCC
count at 1.

Produces:
    P-1 every folder edge in the cycle,
    ranked by folders freed
    P-2 demotion simulation: make folder X a
    pure sink (drop its back-refs)
    P-3 the actual file references behind
    each back-edge, so the work is concrete

Usage:
    python mcd_demotion_plan.py
    <mfs_relations.csv> [--focus Utility] [--
    depth leaf]
    """
    from __future__ import annotations

    import argparse
    import csv
    import re
    from collections import defaultdict
    from pathlib import Path

    FROM_COLS = ("frompath", "from",
```

```

"fromentity", "src_path", "srcpath",
    "source_file", "sourcefile",
"sourcepath", "from_path")
TO_COLS = ("topath", "to", "toentity",
"dst_path", "dstpath",
    "target_file", "targetfile",
"targetpath", "to_path")
ENCODINGS = ("utf-8-sig", "cp949", "utf-8",
"latin-1")

def key(s):
    return re.sub(r"[^a-z0-9]", "", str(s
or "").lower())

def norm(v):
    return str(v or
    "").strip().replace("\\", "/").strip("/")

def load(path: Path):
    files = [path] if path.is_file() else
sorted(path.glob("*.csv"))
    pairs = []
    for f in files:
        rows = None
        for enc in ENCODINGS:
            try:
                with open(f, encoding=enc,
newline="") as fh:
                    rows =
list(csv.reader(fh))
                break

```

```

        except UnicodeDecodeError:
            continue
        except OSError:
            rows = None
        if not rows or len(rows) < 2:
            continue
        head = [key(c) for c in rows[0]]
        fi = next((i for i, c in
enumerate(head) if c in FROM_COLS), None)
        ti = next((i for i, c in
enumerate(head) if c in TO_COLS), None)
        if fi is None or ti is None:
            continue
        for r in rows[1:]:
            if len(r) > max(fi, ti):
                a, b = norm(r[fi]),
norm(r[ti])
                if a and b:
                    pairs.append((a, b))
        return pairs

def tarjan(nodes, edges):
    adj = defaultdict(list)
    for a, b in edges:
        adj[a].append(b)
    idx, low, on, st, out, c = {}, {},
set(), [], [], 0
    for root in sorted(nodes):
        if root in idx:
            continue
        frames = [(root, iter(adj.get(root,
())))]
        idx[root] = low[root] = c

```

```

        c += 1
        st.append(root)
        on.add(root)
    while frames:
        v, it = frames[-1]
        moved = False
        for w in it:
            if w not in idx:
                idx[w] = low[w] = c
                c += 1
                st.append(w)
                on.add(w)
                frames.append((w,
iter(adj.get(w, ())))))
            moved = True
            break
        if w in on:
            low[v] = min(low[v],
idx[w])

        if moved:
            continue
        frames.pop()
        if low[v] == idx[v]:
            comp = set()
            while st:
                w = st.pop()
                on.discard(w)
                comp.add(w)
                if w == v:
                    break
            out.append(comp)
        if frames:
            low[frames[-1][0]] =
min(low[frames[-1][0]], low[v])

```

```

        return out

def in_cycle(edges):
    nodes = {x for e in edges for x in e}
    return {x for c in tarjan(nodes, edges)
            if len(c) > 1 for x in c}

def grouper(depth):
    if depth == "leaf":
        return lambda p: p.rsplit("/", 1)
    [0] if "/" in p else "ROOT"
    d = int(depth)
    return lambda p: "/".join(p.split("/")
                               [:-1][:d]) or "ROOT"

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("csv")
    ap.add_argument("--depth",
                    default="leaf", help="leaf or 1..5")
    ap.add_argument("--focus",
                    default=None, help="substring of a folder
to detail")
    ap.add_argument("--top", type=int,
                    default=15)
    a = ap.parse_args()

    pairs = load(Path(a.csv))
    if not pairs:
        print("no usable rows")
        return

```

```

g = grouper(a.depth)

    behind = defaultdict(list)           #
folder edge -> file references
    fedges = set()
    for src, dst in pairs:
        ga, gb = g(src), g(dst)
        if ga == gb:
            continue
        fedges.add((ga, gb))
        behind[(ga, gb)].append((src, dst))

    base = in_cycle(fedges)
    print("=" * 74)
    print(f"granularity={a.depth}  folders=
{len({x for e in fedges for x in e})}"
          f"  folder edges={len(fedges)}
folders in cycle={len(base)}")

    print("-" * 74)
    print(f"[P-1] single folder edge
removal, ranked by folders freed (top
{a.top})")
    scored = []
    for e in fedges:
        if e[0] not in base or e[1] not in
base:
            continue
        freed = len(base) -
len(in_cycle(fedges - {e}))
        if freed > 0:
            scored.append((freed,
len(behind[e]), e))
    scored.sort(reverse=True)

```



```

    if scored:
        print(f"    {'freed':>6}{ 'refs':>6}
edge")
        for freed, nref, (x, y) in
scored[:a.top]:
            print(f"    {'freed':>6}{nref:>6}
{x} -> {y}")
        else:
            print("    none: no single folder
edge frees any folder."
                " Demotion (P-2) is the
lever.")

    print("-" * 74)
    print("[P-2] demotion: drop ALL back-
references of one folder (make it a sink)")
    outd = defaultdict(set)
    ind = defaultdict(set)
    for x, y in fedges:
        outd[x].add(y)
        ind[y].add(x)
    plans = []
    for f in sorted(base):
        back = {(f, y) for y in outd[f]}
        if not back:
            continue
        freed = len(base) -
len(in_cycle(fedges - back))
        refs = sum(len(behind[e]) for e in
back)
        plans.append((freed, -len(back),
len(ind[f]), len(back), refs, f))
    plans.sort(reverse=True)
    print(f"    {'freed':>6}{ 'cutEdges':>9}

```

```

{'fileRefs':>9}{'dependents':>11} folder")
    for freed, _, nin, nback, refs, f in
plans[:a.top]:
        print(f"    {freed:>6}{nback:>9}
{refs:>9}{nin:>11} {f}")

    if a.focus:
        hit = [f for f in outd if
a.focus.lower() in f.lower()]
        print("-" * 74)
        print(f"[P-3] back-references of
folders matching '{a.focus}'")
        for f in sorted(hit):
            back = sorted(outd[f])
            if not back:
                print(f" {f}: already a
pure sink (no outgoing folder edges)")
                continue
            freed = len(base) -
len(in_cycle(fedges - {(f, y) for y in
back}))
            print(f" {f} out={len(back)}
in={len(ind[f])} "
                f"-> removing all frees
{freed} folders")
            for y in back:
                refs = behind[(f, y)]
                solo = len(base) -
len(in_cycle(fedges - {(f, y)}))
                print(f"    -> {y} file
refs={len(refs)} alone frees {solo}")
                for s, d in refs[:4]:
                    print(f"        {s} -
> {d}")

```

```

        if len(refs) > 4:
            print(f"        ...
{len(refs)-4} more")

    print("=" * 74)
    best_edge = scored[0] if scored else
None
    best_plan = plans[0] if plans else None
    print("COPY THE LINE BELOW AND SEND IT
BACK.")
    print(
        f"ANS8|dep={a.depth}|folders=
{len({x for e in fedges for x in e})}"
        f"|fe={len(fedges)}|incyc=
{len(base)}"
        f"|e1={{str(best_edge[2][0]) + '>'
+ str(best_edge[2][1]) + ':' +
str(best_edge[0])} if best_edge else
'NONE'}}"
        f"|d1={{str(best_plan[5]) + ':' +
str(best_plan[0]) + '/' + str(best_plan[3])
+ '/' + str(best_plan[4])} if best_plan
else 'NONE'}}"
        + "|d2=" + ";".join(f"{p[5]}:
{p[0]}/{p[3]}/{p[4]}" for p in plans[1:4])
    )
    print("(d1/d2 =
folder:foldersFreed/edgesToCut/fileRefsToCh
ange)")
    print("=" * 74)

```

```
if __name__ == "__main__":  
    main()
```

3. STEP 2 – 실행 (2회)

```
python mcd_demotion_plan.py <CSV> --focus  
Utility > plan_utility.txt
```

```
python mcd_demotion_plan.py <CSV> --focus  
Export > plan_export.txt
```

두 번째는 기존 DB 폴더(L1C/Common/Export)의 남은 되참
조 1개를 특정하기 위한 것이다.

출력이 길면 파일로 받아서 필요한 부분만 보면 된다.

4. 회신

(1) ANS8 | 줄 (첫 번째 실행 것)

```
ANS8 |
```

(2) [P-3] 블록 – 이게 실제 작업 목록이다

plan_utility.txt 의 [P-3] 섹션을 붙여넣기. 길면 앞
20줄만.

plan_export.txt 의 [P-3] 섹션도 (짧을 것이다):

(3) 객관식 – Utility 폴더의 성격

- [] 공용 유틸리티 (모두가 쓰는 하위 계층)
- [] 이름만 Utility이고 실제로는 상위 로직 포함
- [] 여러 성격이 섞여 있음
- [] 모름

5. 출력 읽는 법

섹션	내용
[P-1]	폴더 edge 하나를 지우면 몇 개 폴더가 순환에서 빠지는지. refs 는 그 edge 뒤의 실제 파일 참조 수
[P-2]	한 폴더의 되참조를 전부 끊어 싱크로 만들 때 효과. 팀의 DB 폴더 방식과 동일한 계산
[P-3]	지정한 폴더의 되참조를 하나씩 나열 + 그 뒤의 실제 파일 참조. 여기가 작업 지시서다

[P-2] 의 `cutEdges` 는 끊어야 할 폴더 edge 수,
`fileRefs` 는 실제로 손봐야 할 코드 참조 수다.
`fileRefs` 가 작고 `freed` 가 크면 그게 최우선 대상이다.

6. 주의

- 읽기 전용이다. CSV를 수정하지 않는다.
- 기본 granularity는 `leaf` 다. `--depth 2` 등으로 바뀌
가며 볼 수 있으나,
7단계에서 leaf가 가장 구체적이었으므로 leaf 기준으로
판단한다.
- `TOP/L1C`, `TOP/Utility` 같은 이름이 실제 폴더가 아
니라 논리 개념이라면,
[P-3] 에 찍히는 **파일 경로**가 실제 코드 위치다. 그쪽을
기준으로 삼으면 된다.
- 이 계산은 **CSV 위상만** 쓴다. 실제로 끊을 수 있는지(코드
타당성)는 별도 판단이다.